

# **CPP102: PROGRAMMING LOGIC AND DESIGN**

## **Week 3 Assessment — Batch 03**

Assessment and Application of Program Logic Design

Section B | Monday Class

Department of Computer Engineering

## Instructions to Students

Solve all 10 problems below. The problems are divided into three levels of increasing complexity. For every problem, submit all five of the following:

- Input-Process-Output (IPO) Table
- Algorithm (numbered, step-by-step)
- Pseudocode (using START, INPUT, PROCESS, IF, ELSE, DISPLAY, END)
- Flowchart (correct standard symbols)
- Short explanation (1-2 sentences on the logic used)

*Python code is not required at this stage. You are assessed only on your algorithm, pseudocode, and flowchart logic. Python implementation will be introduced in a later lesson.*

Flowcharts may be created using Mermaid, diagrams.net, MS Word shapes, PowerPoint shapes, or hand-drawn and scanned/photographed, as long as correct standard flowchart symbols are used.

*This is Batch 03. The problems below use their own context, numbers, and variable names. Solve only the problems assigned to your batch — do not copy the wording or answers of another batch.*

## Repository and File Organization

Create a repository (or project folder) named:

`cpp102-week3-batch03-lastname`

Replace "lastname" with your actual last name. Inside it, save the following files:

- README.md
- problem01\_algorithm.txt
- problem01\_pseudocode.txt
- problem01\_flowchart.md (or image)
- problem02\_algorithm.txt
- problem02\_pseudocode.txt
- problem02\_flowchart.md (or image)
- problem03\_algorithm.txt
- problem03\_pseudocode.txt
- problem03\_flowchart.md (or image)
- problem04\_algorithm.txt
- problem04\_pseudocode.txt
- problem04\_flowchart.md (or image)
- problem05\_algorithm.txt
- problem05\_pseudocode.txt
- problem05\_flowchart.md (or image)
- problem06\_algorithm.txt
- problem06\_pseudocode.txt

- problem06\_flowchart.md (or image)
- problem07\_algorithm.txt
- problem07\_pseudocode.txt
- problem07\_flowchart.md (or image)
- problem08\_algorithm.txt
- problem08\_pseudocode.txt
- problem08\_flowchart.md (or image)
- problem09\_algorithm.txt
- problem09\_pseudocode.txt
- problem09\_flowchart.md (or image)
- problem10\_algorithm.txt
- problem10\_pseudocode.txt
- problem10\_flowchart.md (or image)
- screenshots/ (folder containing Git evidence screenshots)

README.md must include:

- Student name
- Section
- Batch number: Batch 03
- Class schedule: Monday
- Course: CPP102 — Programming Logic and Design
- Week 3 assessment title: Assessment and Application of Program Logic Design
- List of completed problems (Problem 1 to Problem 10)
- Reflection of at least 3 sentences
- GitHub repository link

## Git Workflow

### First-Time Setup

**1. Initialize the repository.**

```
git init
```

**2. Set your Git commit identity for this repository.**

```
git config user.name "Student Full Name"  
git config user.email "student@email.com"
```

**3. Verify that the identity was set correctly.**

```
git config user.name  
git config user.email
```

**4. Check the current state of your files.**

```
git status
```

**5. Stage all your files.**

```
git add .
```

**6. Commit with a clear message.**

```
git commit -m "Add Week 3 Batch 03 program logic assessment"
```

**7. Set the branch name and connect to GitHub.**

```
git branch -M main  
git remote add origin https://github.com/yourusername/cpp102-week3-  
batch03-lastname.git
```

**8. Push to GitHub.**

```
git push -u origin main
```

### Regular Update Workflow

**1. Always pull first to get the latest version of your repository.**

```
git pull
```

**2. Check the current state of your files.**

```
git status
```

**3. Stage your changes.**

```
git add .
```

**4. Commit with a clear message.**

```
git commit -m "Update Week 3 Batch 03 assessment solutions"
```

**5. Push your changes to GitHub.**

```
git push
```

Take a screenshot after running git status and after a successful git commit. Save both in your screenshots/ folder.

### Reminders

- Always pull before starting an update.
- Always verify your Git name and email before committing on a shared computer.

- Always commit before pushing.
- Avoid editing directly on GitHub unless instructed.

## Assessment Problems

### LEVEL 1: BASIC LINEAR PROBLEMS

*Simple input, process, and output — no decisions.*

#### Problem 1: Auditorium Seat Count

An auditorium's seating is arranged in equal rows and columns. Create a logic design that asks for the number of rows and the number of columns, computes the total number of seats, and displays the result.

#### Problem 2: Fence Around a Lot

A square lot needs to be fenced on all sides. Create a logic design that asks for the length of one side, computes the perimeter, and displays the result.

#### Problem 3: Jeepney Fare Computation

A jeepney charges a fixed rate for every kilometer traveled. Create a logic design that asks for the rate per kilometer and the distance traveled, computes the total fare, and displays the result.

#### Problem 4: Examination Average

A student has three examination scores. Create a logic design that asks for the three scores, computes the average, and displays the result.

## LEVEL 2: MULTIPLE-INPUT COMPUTATION PROBLEMS

*Several inputs combined through more than one arithmetic operation — still no decisions.*

### Problem 5: Online Shopping Cart Total

An online shopper adds three different items to a cart, each with its own price and quantity. Create a logic design that asks for the price and quantity of each of the three items, computes the total cost, and displays the result.

### Problem 6: Total Travel Cost

A family trip involves a fare, a toll fee, and a fuel cost. Create a logic design that asks for the fare, the toll fee, and the fuel cost, computes the total travel cost, and displays the result.

### Problem 7: Semester Standing Computation

A student's semester standing is based on a quiz average, an examination score, and a project score, weighted at 25%, 45%, and 30% respectively. Create a logic design that asks for the three scores, computes the semester standing, and displays the result.

## LEVEL 3: DECISION PROBLEMS

*IF/ELSE logic producing two or three possible outputs.*

### Problem 8: Senior Citizen Discount Eligibility

A restaurant gives a senior citizen discount to customers who are 60 years old or above. Create a logic design that asks for a customer's age and displays "Eligible for Senior Discount" if the age is 60 or above, or "Not Eligible for Senior Discount" otherwise.

### Problem 9: Elevation Reading Classification

A hiking app records elevation relative to sea level. Create a logic design that asks for an elevation reading and displays "Above Sea Level" if it is greater than 0, "Below Sea Level" if it is less than 0, or "At Sea Level" if it is exactly 0.

### Problem 10: Bus Seat Number Check

Bus seats are numbered for an out-of-town field trip. Create a logic design that asks for a seat number and displays "Even" if the number is divisible by 2, or "Odd" otherwise.

## Grading Rubric (100 Points)

| Criterion                                          | Full Marks                                                                                                                                                                                    | Partial Marks                                                                                                                                            | No Credit                                                                                                           |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Input-Process-Output Analysis (15 pts)             | Full (15 pts): All 10 problems have complete, correct IPO tables — inputs, process, and output are all clearly and correctly identified.                                                      | Partial (7-8 pts): Most IPO tables are present and mostly correct, but a few problems have incomplete or minor errors.                                   | No Credit (0 pts): IPO tables are missing, or input/process/output are consistently incorrect or absent.            |
| Algorithm Correctness and Completeness (20 pts)    | Full (20 pts): All 10 algorithms are correctly numbered, logically complete from Start to End, and correctly solve each problem, including proper decision steps where required.              | Partial (10 pts): Most algorithms are logically correct but have minor gaps, missing steps, or small logic errors in one or two problems.                | No Credit (0 pts): Algorithms are missing, incomplete, or do not solve the given problems.                          |
| Pseudocode Correctness and Readability (20 pts)    | Full (20 pts): All 10 pseudocode solutions correctly use the required keywords (START, INPUT, PROCESS, IF, ELSE, DISPLAY, END), are properly indented, and match the corresponding algorithm. | Partial (10 pts): Pseudocode is present for most problems but has inconsistent keyword usage, indentation issues, or does not fully match the algorithm. | No Credit (0 pts): Pseudocode is missing or does not follow the required keywords and structure.                    |
| Flowchart Logic and Correct Symbol Usage (25 pts)  | Full (25 pts): All 10 flowcharts use correct standard symbols (terminal, input/output, process, decision), show correct logic flow, and match the algorithm and pseudocode.                   | Partial (12 pts): Most flowcharts are present and mostly correct, but a few use incorrect symbols or contain minor logic errors.                         | No Credit (0 pts): Flowcharts are missing, use incorrect symbols throughout, or do not represent the correct logic. |
| Git/GitHub Evidence and File Organization (10 pts) | Full (10 pts): Repository is correctly named, all required files are present and properly organized, and screenshots clearly show git status and a successful commit.                         | Partial (5 pts): Repository and files are present but incompletely organized, or a required screenshot is missing.                                       | No Credit (0 pts): No repository, missing files, or no evidence of Git usage.                                       |
| Completeness, Neatness, and                        | Full (10 pts): All 10 problems are                                                                                                                                                            | Partial (5 pts): Some problems or files are                                                                                                              | No Credit (0 pts): Submission is largely                                                                            |

| Criterion                     | Full Marks                                                                                                                                   | Partial Marks                                                                                     | No Credit                                             |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| README<br>Reflection (10 pts) | complete, files are neatly organized and labeled, and README.md contains all required information plus a reflection of at least 3 sentences. | missing or disorganized, or the README is incomplete, or the reflection is shorter than required. | incomplete or disorganized, or the README is missing. |

Total: 100 points.